

Mathitude — Payment Compliance

Ari · for Paula · 2026-05-24

May 24, 2026

CONTENTS

Mathitude — Payment data handling & PCI posture	
1. The card never touches Mathitude's servers	
2. The data model — one card per parent	
3. What Mathitude stores	
4. What Stripe stores	
5. Network path of a card number	
6. Access controls	
Admin tiers (5/24)	
Stripe key scoping	
Webhook signature verification	
7. Logging hygiene	
8. Data residency	
9. Quick mental model for Paula	
10. SOC 2 path (when Mathitude wants its own attestation)	
11. Sources + repo references	
Sign-off summary	

Mathitude — Payment data handling & PCI posture

Date: 2026-05-24 (revised) **Audience:** Paula + anyone asking how cards are stored — book-keepers, accountants, lawyers, parents who want assurance. **TL;DR:** Mathitude never sees, never transmits, and never stores credit card numbers. All card data goes from the customer's browser **directly to Stripe** over an encrypted channel inside a Stripe-hosted iframe. Mathitude only ever holds Stripe's tokenized references (`pm_...` , `cus_...`) and a brand + last-4 for display. Stripe is PCI DSS Level 1 — the highest tier of card-handler compliance.

1. The card never touches Mathitude's servers

When a parent (or Paula on a parent's behalf, on `/admin/families/[id]`) sees a card form on the Mathitude portal, the input field is **not Mathitude's HTML**. It is a Stripe-hosted iframe rendered by `@stripe/react-stripe-js`'s `<CardElement>` component (`src/components/stripe/save-card-form.tsx`).

When the user types their card number, expiry, CVC, and ZIP, those keystrokes are captured inside Stripe's iframe DOM. Our JavaScript **cannot read them** — the iframe is served from a different origin (`js.stripe.com`) and the browser's same-origin policy isolates it from the surrounding Mathitude page.

On submit, our code calls `stripe.confirmCardSetup(clientSecret, ...)`. Stripe's JS reads the card values from its own iframe, encrypts them, and POSTs them **directly to `api.stripe.com`** over TLS 1.2+. Stripe returns a `PaymentMethod` token (`pm_...`) to our browser-side code.

Our code then calls our own server (`POST /api/stripe/payment-methods/finalize-new-card`) with **only the `paymentMethodId` and `parentId`**. The card number, expiry, and CVC have never crossed Mathitude's network.

Our server attaches the `paymentMethod` to the parent's Stripe `Customer` (`cus_...`) via Stripe's Node SDK using our restricted secret key. That's the entire path.

This is Stripe's standard **Elements + SetupIntent** pattern. It is the flow Stripe recommends for SAQ A PCI eligibility — the lowest-burden PCI Self-Assessment Questionnaire because the merchant never handles cardholder data.

2. The data model — one card per parent

Updated 5/24 (Paula's clarification):

- **Each parent** on a family has their own Stripe `Customer` record.
- **Each parent's customer has exactly one card on file.** Saving a new card replaces the previous card for that parent (enforced both at card-save time and via the Stripe webhook for redundancy).
- **A family can have multiple parents**, so a family can have multiple cards available — one belonging to mom, one belonging to dad, one belonging to a grandparent, etc.
- **The family record points at one parent as `primaryPayerId`**. That parent's card is what gets charged. To switch which card is billed, Paula clicks "Make primary payer" on the parent whose card she wants charged.

This matches the real-world flow Paula described: husband adds his card under Parent 2; the family stays billed to Parent 1 until Paula explicitly switches the primary payer.

3. What Mathitude stores

In DynamoDB (`mathitude-staging-*` tables in AWS region `us-west-2`, account `050451400186`, all tables encrypted at rest via AWS-managed KMS keys, point-in-time recovery enabled):

Table	What we store	What we never store
<code>parents</code>	Parent's name, email, phone, family link, Stripe customer reference <code>stripeCustomerId</code> (e.g. <code>cus_R8xQ...</code>)	Card number, expiry, CVC
<code>families</code>	<code>primaryPayerId</code> pointer to the parent currently billed	Card data
<code>payments</code>	Stripe <code>paymentIntent</code> + <code>charge</code> ids, amount in cents, status, brand+last4 (display only)	Card number, expiry, CVC, ZIP
<code>notifications</code>	Activity log entries with brand + last4 for human-readable display ("Visa ending in 7710")	Card number, expiry, CVC
<code>secrets</code>	Stripe restricted API key + webhook signing secret; admin email list	Anything cardholder-related

The four-digit last-4 + card brand are explicitly defined by PCI DSS as **non-sensitive** — they are insufficient on their own to charge a card. Stripe returns them to us alongside the PaymentMethod token; we display them so Paula can identify which card a customer is using (“Visa ending in 7710”).

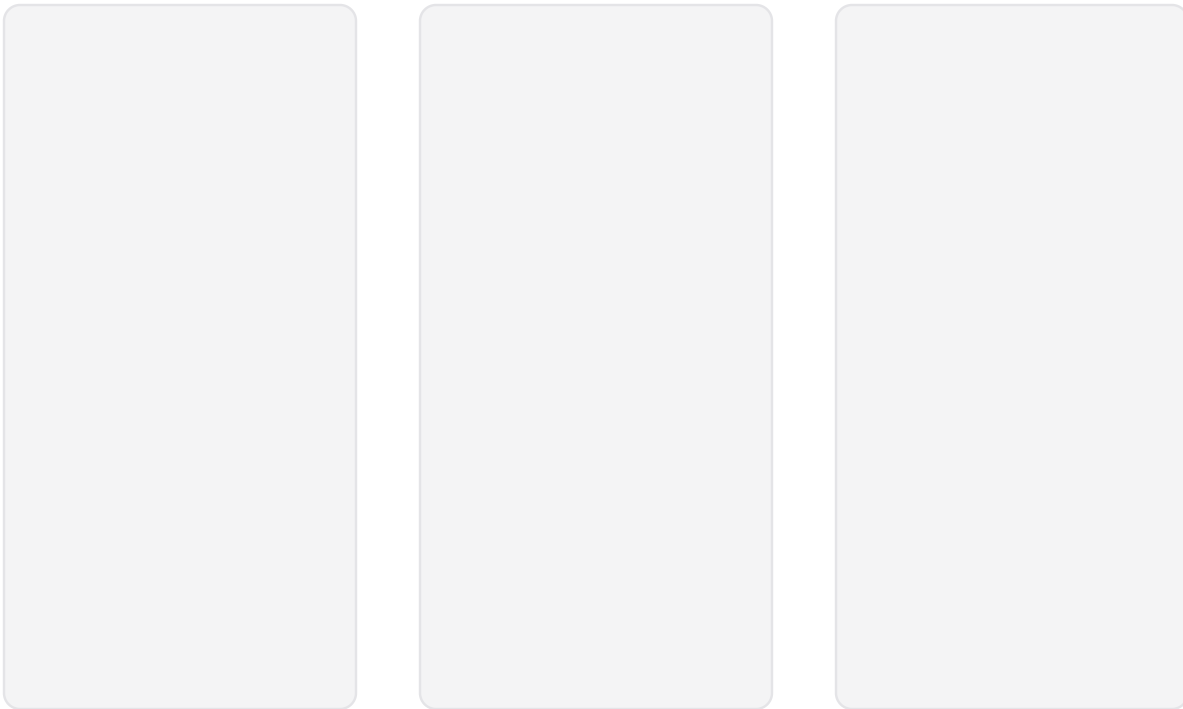
4. What Stripe stores

Stripe (PCI DSS Level 1, SOC 2 Type 2) holds:

- Full card number (PAN)
- Card expiry month + year
- CVV/CVC (only momentarily during the initial card-verification charge; Stripe never stores CVC long-term, per PCI rules)
- Cardholder name + billing ZIP
- Linkage between Stripe `customer` → `paymentMethod`

Stripe’s attestation of PCI compliance is downloadable from their dashboard under Compliance → Documents. SOC 1 / SOC 2 reports are also available there.

5. Network path of a card number



Parent's browser Stripe (PCI Level 1) Mathitude

PCI SCOPE — STRIPE-MANAGED Mathitude page (UI) our JS, can't read card `<CardElement />` Stripe-hosted iframe (`js.stripe.com`) parent types card here `api.stripe.com` Stripe vault stores PAN + expiry `/api/stripe/...` our Next.js server never sees PAN DynamoDB stores `pm_...` and `cus_...` tokens only PAN over TLS 1.2+ `pm_...` token `pm_...` HTTPS — token + parentId only no card number on this hop attach `pm_...` to `cus_...` `cus_...`, `pm_...` Card data path (Stripe-handled) Token-only path (Mathitude-handled)

Reading the diagram. The dashed purple boundary shows where the card number lives: in the Stripe-hosted iframe and on Stripe's servers, never elsewhere. Two arrows ever carry card data (the purple ones); both endpoints are Stripe. Every black arrow carries only Stripe tokens (`pm_...`, `cus_...`) — those are what Mathitude's servers and DynamoDB store. There is no path on this diagram by which a card number reaches Mathitude's infrastructure.

Mathitude's servers **never appear on the card-number path**. There is no configuration setting Paula or any future operator can change to make card numbers traverse our infrastructure — the Stripe iframe enforces this at the browser level.

6. Access controls

ADMIN TIERS (5/24)

Two roles enforced at the application layer (`src/lib/server/admins.ts`):

- **Master admin** — can do everything, including adding/removing other admins and changing their roles. Bootstrap master admins (Paula, Ari, Nikki) are baked into the codebase and cannot be removed via the UI.
- **Admin** — full operator access to families, students, sessions, billing, payments, financials. **Cannot add or remove other admins**; the admin-management endpoints return 403 with `code: "not_master_admin"` if a plain admin attempts mutation.

Both tiers are gated behind Clerk authentication. There is no way to access the admin portal without first signing in with an email on the admin list.

STRIPE KEY SCOPING

Our backend uses a Stripe **restricted key** (`rk_...`) where possible (`src/lib/server/stripe.ts:153`). The key is scoped to the minimum permissions our flows require:

- `paymentmethods:write` (attach/detach to customers)
- `customers:write` (create on first card save, update default)
- `paymentintents:write` (charge approval queue)
- `setupintents:write` (save-card flow)

The key lives in DynamoDB (`mathitude-staging-secrets`), encrypted at rest, never echoed in any response — `getStripeMeta()` only returns `last4` of the key for diagnostic display in `/admin/settings/stripe`).

WEBHOOK SIGNATURE VERIFICATION

`POST /api/stripe/webhook` verifies every incoming Stripe webhook via the `Stripe-Signature` header against the webhook signing secret. Unsigned or stale-timestamp webhooks are rejected with 400. This prevents an attacker from sending forged `payment_intent.succeeded` events to mark debts as paid.

7. Logging hygiene

`grep -r "cardNumber\|card_number\|pan" src/` returns zero hits across the codebase as of this date. Stripe error objects sometimes include the last-4 of a declined card; those are safe to log per PCI rules. Application logs (Vercel, AWS CloudWatch) contain no PANs.

8. Data residency

- **DynamoDB:** AWS `us-west-2` (Oregon)
- **Stripe:** see Stripe's data-residency commitments at https://stripe.com/docs/security/stripeS-MARTPANTS_PRESERVED_386 — [primary processing in the US for US customers](#)
- **Clerk (auth):** see Clerk's [processor agreement](#)
- **Resend (email):** see Resend's [DPA](#)
- **Vercel (host):** see Vercel's [DPA](#)

[For European customers, this stack would need a residency review. As of today, Mathitude operates in California.](#)

9. Quick mental model for Paula

If a bookkeeper, accountant, or lawyer ever asks “is Mathitude storing credit cards somewhere?” — the honest answer is **no**. The cards live in Stripe's vault. We have a key-card to the vault (the restricted Stripe API key) and a list of who has which card (the `payment-Method` and `customer` ids). We do not have the cards themselves. We could not give card-holder data to a hacker, a subpoena, or anyone else if asked, because we do not possess it.

10. SOC 2 path (when Mathitude wants its own attestation)

[Stripe + Clerk + AWS are all SOC 2 Type 2 themselves, so most of the heavy lifting on payment + auth controls is inherited from these sub-processors. Mathitude-side controls for a Type 1 attestation are mostly about access logs, encryption verification, vendor management, and an incident-response runbook.](#)

[Cheapest first step: enable AWS CloudTrail + retain logs 365 days; turn on AWS GuardDuty. Costs <\\$50/mo and is required evidence for any future SOC 2 audit.](#)

Full path: pick a compliance vendor (Vanta, Drata, Secureframe), ~\$10– 15K/year. They auto-collect controls from AWS + Vercel + Clerk + Stripe. ~30 days of evidence collection, then auditor review. Type 1 attestation in ~3 months. Type 2 attestation needs 6+ months of evidence after that.

11. Sources + repo references

External:

- [Stripe Elements docs](#)
- [Stripe PCI compliance overview](#)
- PCI DSS v4.0 §3.3 + §3.5 (tokenization scope reduction)

Repo files (verifiable in source):

- `src/components/stripe/save-card-form.tsx` — Stripe Elements card form
 - `src/app/api/stripe/create-setup-intent/route.ts` — server-side SetupIntent + on-demand customer creation
 - `src/app/api/stripe/payment-methods/finalize-new-card/route.ts` — single-card enforcement on save
 - `src/app/api/stripe/payment-methods/apply/route.ts` — bulk apply endpoint for the Save Changes flow
 - `src/app/api/stripe/webhook/route.ts` — signed Stripe webhook handler
 - `src/lib/server/stripe.ts` — `enforceSingleCardForCustomer`, `ensureDefaultCard`, restricted-key resolution
 - `src/lib/server/secrets.ts` — Stripe key storage (encrypted at rest, portal-editable, env-var fallback)
 - `src/lib/server/admins.ts` — master-admin vs admin tier enforcement
-

Sign-off summary

Concern	Status
Do we see card numbers?	No. Stripe iframe blocks our JS from reading.
Do we transmit card numbers to our server?	No. Only <code>pm_...</code> tokens cross our network.
Do we store card numbers?	No. Only Stripe ids + brand + last-4.
Is Stripe PCI-compliant?	Yes, Level 1 — the top tier.
Are admin actions authenticated?	Yes. Clerk + master-admin role enforcement.
Are webhooks signature-verified?	Yes. <code>STRIPE_WEBHOOK_SECRET</code> checked on every event.
Encryption at rest for our data?	Yes. DynamoDB tables use AWS-managed KMS.
Can Mathitude leak card data to a hacker?	No. We don't have the data to leak.

Document maintainer: Ari (ari@coframe.com). For the latest version, check `COMPLIANCE.md` in the repo root.